

Analisi di Dati sui Voli Aerei

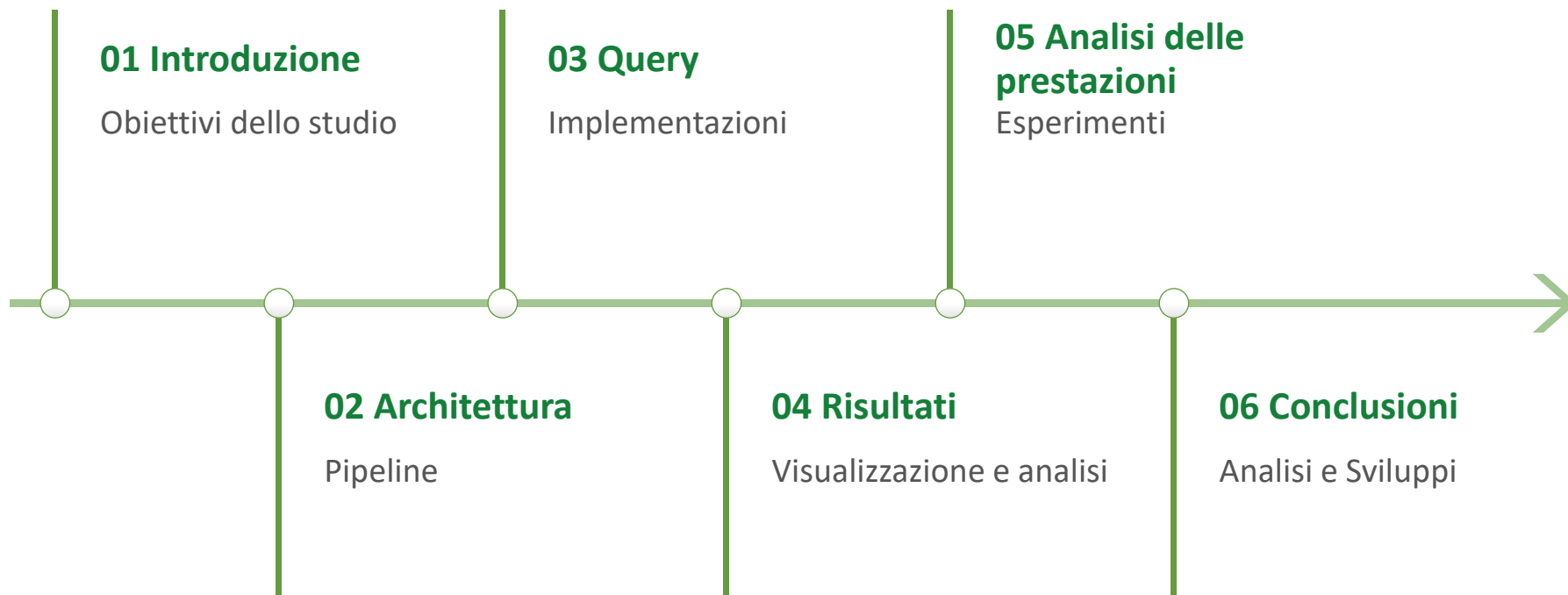
con Apache Spark

Pipeline di Batch Processing su Architettura Containerizzata

SABD A.A. 2025/26 | Prof. Cardellini & Nardelli

Valentina Jin | Elisa Marzioli

Ingegneria Informatica – Università degli Studi di Roma Tor Vergata



01 Introduzione

Il dominio

- **Batch Processing** su dati di volo del Bureau of Transportation Statistics degli Stati Uniti (BTS);
- **Circa 2,2 mln** di voli da **Gennaio 2025** a **Aprile 2025**;



Obiettivi dello studio

Q1

Statistiche mensili di ritardo e cancellazione per AA e DL

Q2

Top-10 compagnie per ritardo medio in arrivo con breakdown delle cause

Q3

Percentili P25/P50/P75/P90 di DEP_DELAY per ora e compagnia (AA, DL, UA, WN)

E1

DataFrame vs RDD

E2

Warm start vs Cold start

E3

Scalabilità del numero di worker

E4

Impatto del caching

02 Architettura

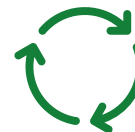


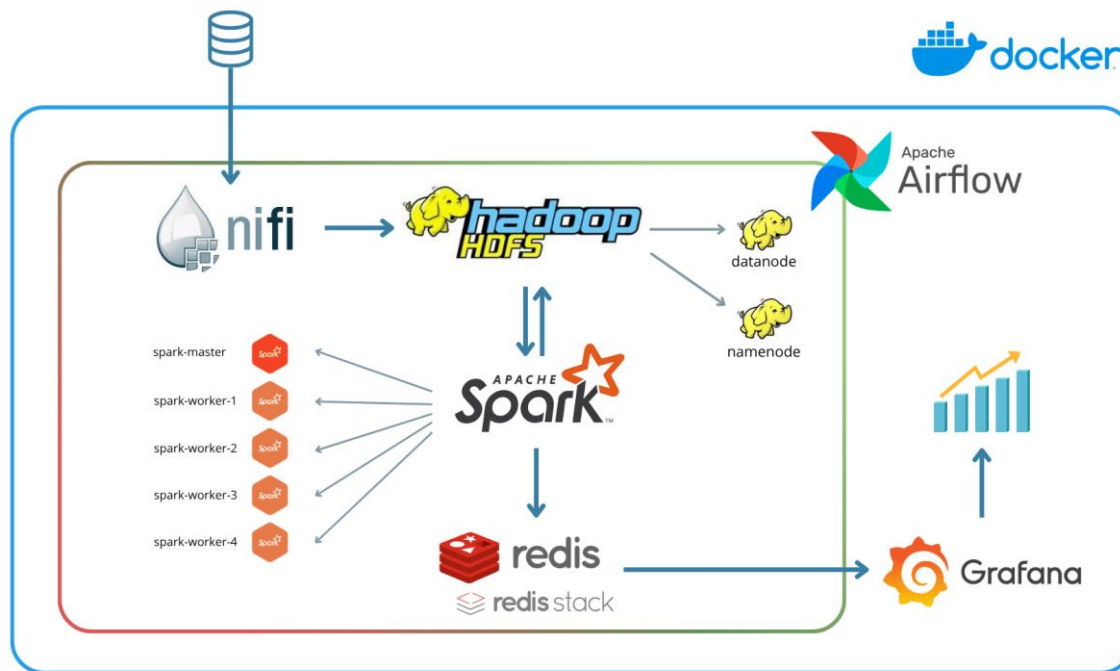
Dati

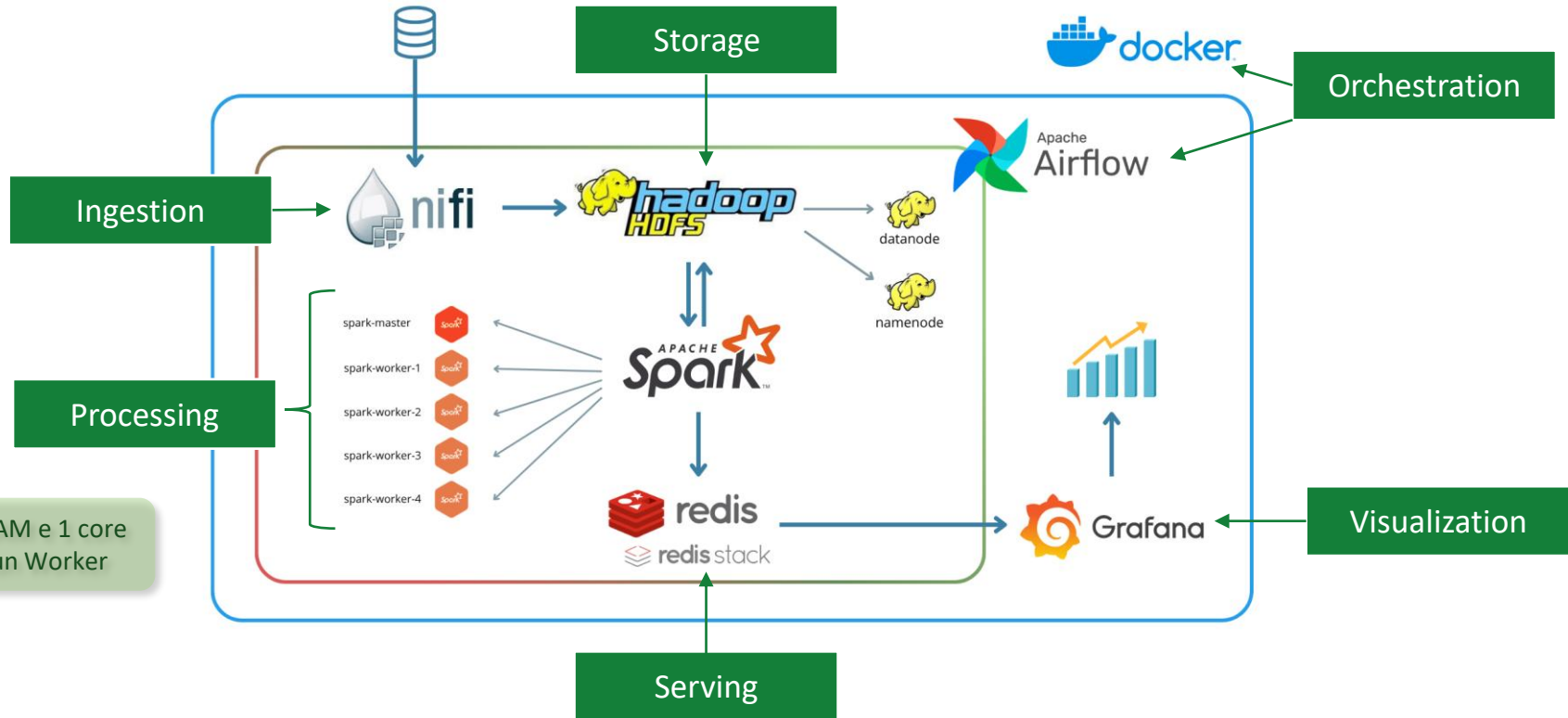
- **Parquet:** formato storage su HDFS.
- **DataFrame:** API principale per il processamento.
- **RDD:** API alternativa per il confronto.
- **CSV:** formato di output dei risultati.

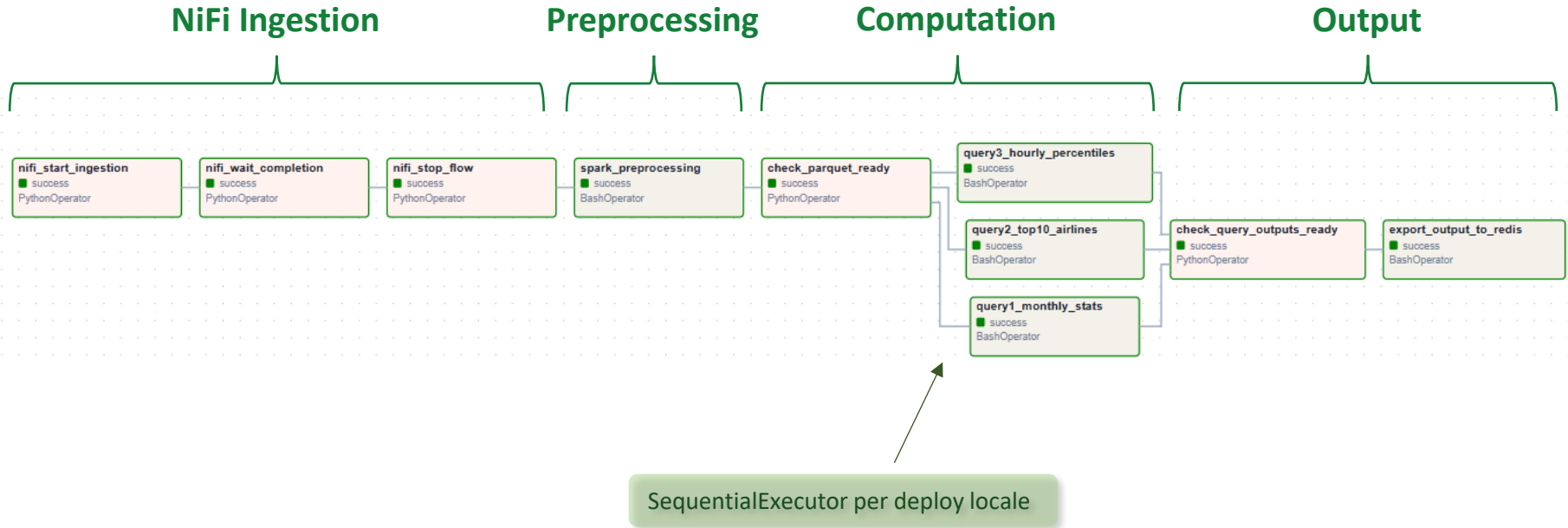
Preprocessing and Processing

- **Preprocessing** eseguito una sola volta per tutte le query, in Spark.
- Selezione delle **colonne utili** alle tre query.
- **Eliminazione** di righe con OP_UNIQUE_CARRIER nullo.
- I valori **null** sono stati settati a 0.
- Scrittura in formato **Parquet non partizionato**.

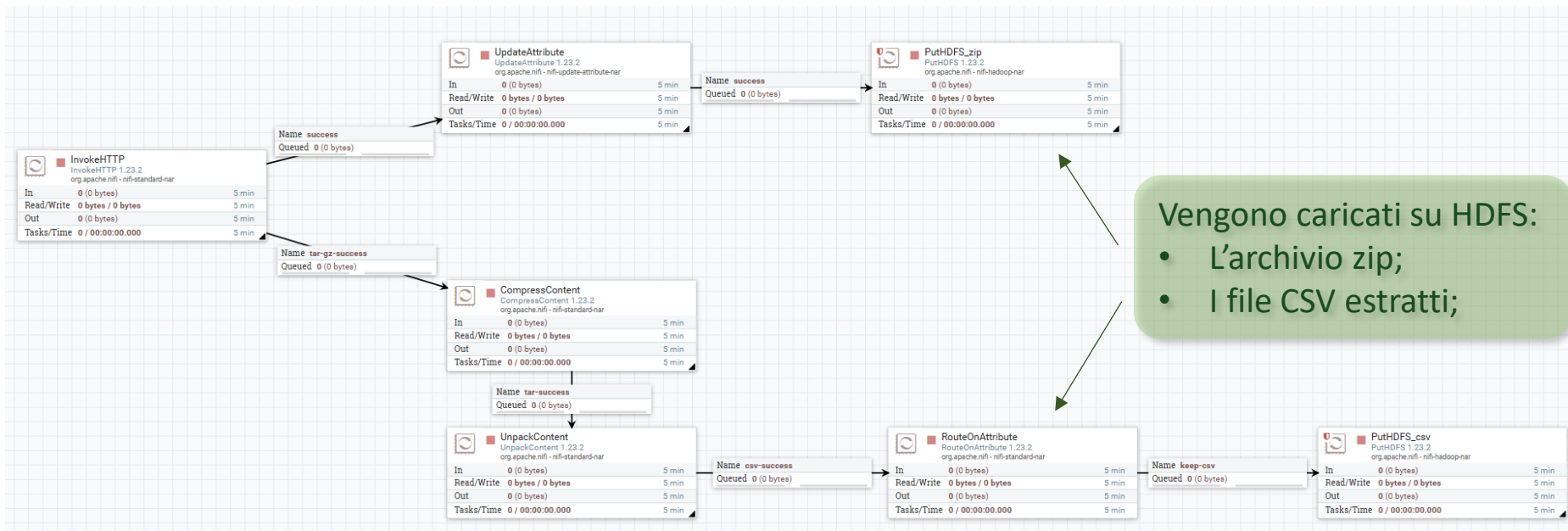








Versione NiFi: 1.23.2 (compatibilità HDFS)



03 Query

Q1 – Statistiche Mensili per Compagnia (AA · DL)): mean/min/max di DEP_DELAY e ARR_DELAY + CANCELLATION_RATE per ogni mese

```
df = (  
    spark.read.parquet(PARQUET_PATH)  
    .select(  
        "MONTH",  
        "OP_UNIQUE_CARRIER",  
        "DEP_DELAY",  
        "ARR_DELAY",  
        "CANCELLED",  
    )  
)  
  
df_filtered = (  
    df.filter(F.col("OP_UNIQUE_CARRIER").isin(TARGET_AIRLINES))  
)
```

Column pruning

```
result = (  
    df_filtered  
    .groupBy("MONTH", "OP_UNIQUE_CARRIER")  
    .agg(  
        F.count("*").alias("total_flights"),  
        F.sum(is_cancelled).alias("cancelled_flights"),  
  
        F.avg(F.when(is_not_cancelled, F.col("DEP_DELAY"))).alias("avg_dep_delay"),  
        F.min(F.when(is_not_cancelled, F.col("DEP_DELAY"))).alias("min_dep_delay"),  
        F.max(F.when(is_not_cancelled, F.col("DEP_DELAY"))).alias("max_dep_delay"),  
  
        F.avg(F.when(is_not_cancelled, F.col("ARR_DELAY"))).alias("arr_delay_mean"),  
        F.min(F.when(is_not_cancelled, F.col("ARR_DELAY"))).alias("arr_delay_min"),  
        F.max(F.when(is_not_cancelled, F.col("ARR_DELAY"))).alias("arr_delay_max"),  
    )  
    .withColumn(  
        "cancellation_rate", F.col("cancelled_flights") / F.col("total_flights") * 100  
    )  
)
```

Q1 – Statistiche Mensili per Compagnia (AA · DL): mean/min/max di DEP_DELAY e ARR_DEPLY + CANCELLATION_RATE per ogni mese

```
rdd_base = (  
    spark.read.parquet(PARQUET_PATH)  
    .select("MONTH", "OP_UNIQUE_CARRIER", "DEP_DELAY",  
"ARR_DELAY", "CANCELLED")  
    .rdd  
    .map(lambda row: (  
        row["OP_UNIQUE_CARRIER"],  
        row["MONTH"],  
        row["DEP_DELAY"],  
        row["ARR_DELAY"],  
        row["CANCELLED"], ))  
    .filter(lambda r: r[0] in TARGET_AIRLINES)  
    .cache()  
)
```

combiner = (sum, min, max, count)

Ogni partizione aggrega localmente, poi Spark unisce i risultati parziali durante lo shuffle.

```
dep_stats = (  
    rdd_base  
    .filter(lambda r: r[4] == 0 and r[2] is not None)  
    .map(lambda r: ((r[1], r[0]), r[2]))  
    .combineByKey(create_combiner, merge_value, merge_combiners)  
    .mapValues(to_stats)  
)  
arr_stats = (  
    rdd_base  
    .filter(lambda r: r[4] == 0 and r[3] is not None)  
    .map(lambda r: ((r[1], r[0]), r[3]))  
    .combineByKey(create_combiner, merge_value, merge_combiners)  
    .mapValues(to_stats)  
)  
cancel_stats = (  
    rdd_base  
    .map(lambda r: ((r[1], r[0]), (r[4], 1)))  
    .reduceByKey(lambda a, b: (a[0]+b[0], a[1]+b[1]))  
    .mapValues(lambda v: round((v[0]/v[1])*100.0, 4))  
)
```

Key = (month, carrier), metric

Accumulatore per voli cancellati

Q2 – Ritardi medi e cause per compagnia + Top 10 compagnie aeree per ritardo medio

```
df_filtered= (  
  df  
  .filter((F.col("CANCELLED") == 0) & (F.col("DIVERTED") == 0))  
  .filter(F.col("ARR_DELAY").isNotNull())  
)
```

Statistiche medie

```
all_airlines_stats = (  
  df_filtered  
  .groupBy(F.col("OP_UNIQUE_CARRIER").alias("carrier"))  
  .agg(  
    F.count(F.lit(1)).alias("num_flights"),  
    F.avg("ARR_DELAY").alias("arrdelay_mean"),  
    F.avg("CARRIER_DELAY").alias("carrier_delay_mean"),  
    F.avg("WEATHER_DELAY").alias("weather_delay_mean"),  
    F.avg("NAS_DELAY").alias("nas_delay_mean"),  
    F.avg("SECURITY_DELAY").alias("security_delay_mean"),  
    F.avg("LATE_AIRCRAFT_DELAY").alias("late_aircraft_delay_mean"),  
  )  
  .orderBy("carrier")  
)
```

Selezione Top 10

```
top10_arrival_delay = (  
  all_airlines_stats  
  .filter(F.col("num_flights") >= MIN_VALID_FLIGHTS)  
  .orderBy(F.col("arrdelay_mean").desc())  
  .limit(10)  
)
```

Q2 – Classifica compagnie aeree per ritardo medio

Statistiche medie

```
all_airlines_rdd = (  
    valid_flights  
    .map(lambda r: (  
        r[0],  
        (  
            r[3], # ARR_DELAY  
            r[4], # CARRIER_DELAY  
            r[5], # WEATHER_DELAY  
            r[6], # NAS_DELAY  
            r[7], # SECURITY_DELAY  
            r[8], # LATE_AIRCRAFT_DELAY  
        )  
    ))  
    .combineByKey(create_combiner, merge_value, merge_combiners)  
    .map(to_result_row)  
    .sortBy(lambda r: r[0])  
    .cache()  
)
```

Calcola la media

Selezione Top 10

```
top10_rdd = (  
    all_airlines_rdd  
    .filter(lambda r: r[1] >= MIN_FLIGHTS_FOR_RANKING)  
    .sortBy(lambda r: -r[2])  
    .zipWithIndex()  
    .filter(lambda kv: kv[1] < 10)  
    .map(lambda kv: kv[0])  
    .cache()  
)
```

Ogni partizione aggrega localmente, poi Spark unisce i risultati parziali durante lo shuffle.

combiner = (arr, carrier, weather, nas, security, late)

Q3 – Percentili P25/P50/P75/P90 di DEP_DELAY per ora e compagnia (AA, DL, UA, WN) + min/max DEP_DELAY

```
df = (spark.read.parquet(PARQUET_PATH)
      .select(
        "OP_UNIQUE_CARRIER",
        "CRS_DEP_TIME",
        "DEP_DELAY",
        "CANCELLED",
      )
)

df_filtered = (
  df
  .filter(F.col("OP_UNIQUE_CARRIER").isin(TARGET_AIRLINES))
  .filter(F.col("CANCELLED") == 0)
  .filter(F.col("DEP_DELAY").isNotNull())
  .withColumn(
    "HOUR",
    (F.col("CRS_DEP_TIME") / 100).cast(IntegerType())
  )
  .filter((F.col("HOUR") >= 0) & (F.col("HOUR") <= 23))
  .cache()
)
```

Calcolo di min/max

```
result_minmax = (
  df_filtered
  .groupBy(
    F.col("OP_UNIQUE_CARRIER").alias("airline"),
  )
  .agg(
    F.min("DEP_DELAY").alias("min_delay"),
    F.max("DEP_DELAY").alias("max_delay"),
  )
)
```

Q3 – Percentili P25/P50/P75/P90 di DEP_DELAY per ora e compagnia (AA, DL, UA, WN) + min/max DEP_DELAY

Calcolo dei percentili

```
result_percentiles = (  
    df_filtered  
    .groupBy(  
        F.col("OP_UNIQUE_CARRIER").alias("airline"),  
        F.col("HOUR").alias("hour"),  
    )  
    .agg(  
        F.count(F.lit(1)).alias("num_flights"),  
        F.percentile_approx("DEP_DELAY", 0.25, PERCENTILE_ACCURACY).alias("p25"),  
        F.percentile_approx("DEP_DELAY", 0.50, PERCENTILE_ACCURACY).alias("p50"),  
        F.percentile_approx("DEP_DELAY", 0.75, PERCENTILE_ACCURACY).alias("p75"),  
        F.percentile_approx("DEP_DELAY", 0.90, PERCENTILE_ACCURACY).alias("p90"),  
    )  
)
```

Scelta dell'algoritmo:

percentile_approx è la funzione nativa di Spark che implementa l'algoritmo **Greenwald-Khanna** per il calcolo approssimato dei percentili. Beneficia delle ottimizzazioni offerte dal **Catalyst Optimizer**.

Vantaggi:

- Nessun trasferimento dei dati al driver
- Accuracy=1000 → errore relativo $\leq 0.1\%$
- Restituisce valori osservati nel dataset (interi)

Q3 – Percentili P25/P50/P75/P90 di DEP_DELAY per ora e compagnia (AA, DL, UA, WN) + min/max DEP_DELAY

```
rdd_base = (  
    spark.read.parquet(PARQUET_PATH)  
    .select("OP_UNIQUE_CARRIER",  
            "CRS_DEP_TIME", "DEP_DELAY", "CANCELLED")  
    .rdd  
    .map(lambda row: (  
        row["OP_UNIQUE_CARRIER"],  
        row["CRS_DEP_TIME"],  
        row["DEP_DELAY"],  
        row["CANCELLED"],  
    ))  
    .filter(lambda r: r[0] in target)  
    .filter(lambda r: r[3] == 0)  
    .filter(lambda r: r[2] is not None)  
    .filter(lambda r: r[1] is not None)  
    .map(lambda r: (r[0], int(r[1]) // 100, r[2]))  
    .filter(lambda r: 0 <= r[1] <= 23)  
    .cache()  
)
```

Calcolo di min/max

```
rows_minmax = (  
    rdd_base  
    .map(lambda r: (r[0], r[2]))  
    .combineByKey(  
        lambda v: (v, v),  
        lambda acc, v: (min(acc[0], v), max(acc[1], v)),  
        lambda a, b: (min(a[0], b[0]), max(a[1], b[1]))  
    )  
    .map(lambda kv: (kv[0], round(kv[1][0], 4), round(kv[1][1], 4)))  
)
```

Q3 – Percentili P25/P50/P75/P90 di DEP_DELAY per ora e compagnia (AA, DL, UA, WN) + min/max DEP_DELAY

Calcolo dei percentili

```
groups_with_digest = (  
    rdd_base  
    .mapPartitions(_build_digests_partition)  
    .reduceByKey(merge_digests)  
    .collect()  
)  
  
rows_percentiles = []  
for (airline, hour), td in groups_with_digest:  
    p25 = round(td.percentile(25), 4)  
    p50 = round(td.percentile(50), 4)  
    p75 = round(td.percentile(75), 4)  
    p90 = round(td.percentile(90), 4)  
    rows_percentiles.append((airline, hour, n, p25, p50, p75, p90))
```

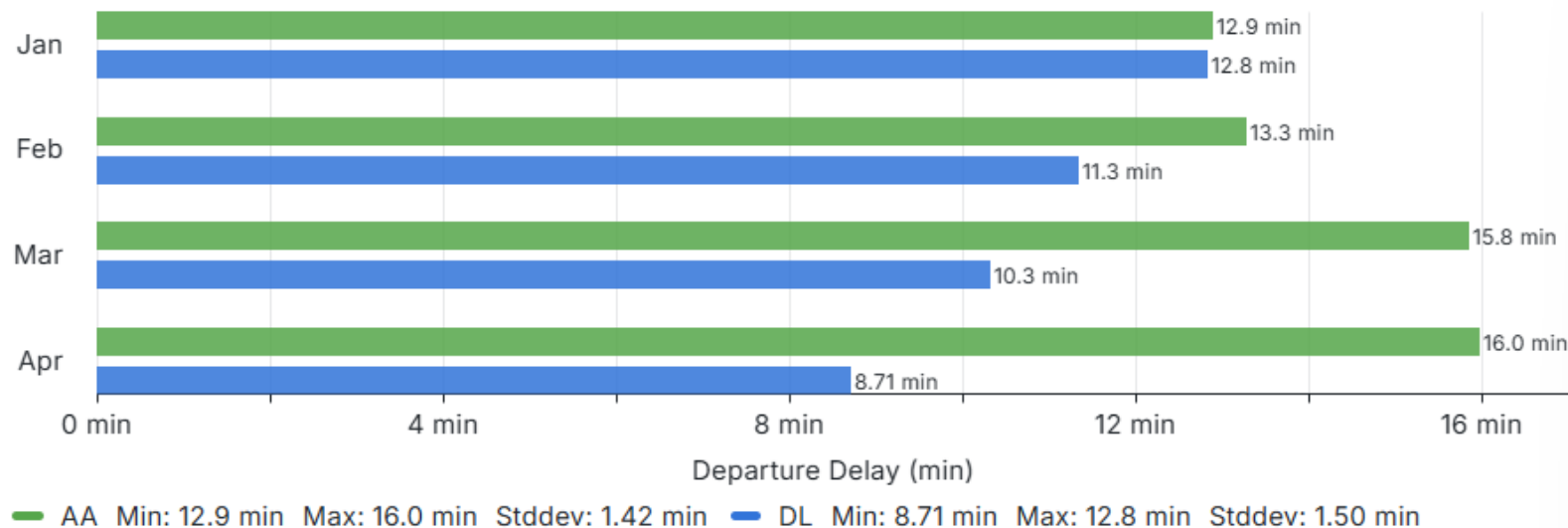
Scelta dell'algoritmo t-digest:

T-digest è un algoritmo probabilistico per il calcolo di percentili approssimati su grandi dataset in modo distribuito. La sua struttura interna è compatibile con le operazioni RDD (pattern **mapPartitions** → **reduceByKey**), replicando in modo esplicito ciò che `percentile_approx` fa internamente nei DataFrame.

- Più accurato alle code (P90)
- Restituisce valori interpolati (decimali)
- Differenza max < 1 minuto vs `percentile_approx`

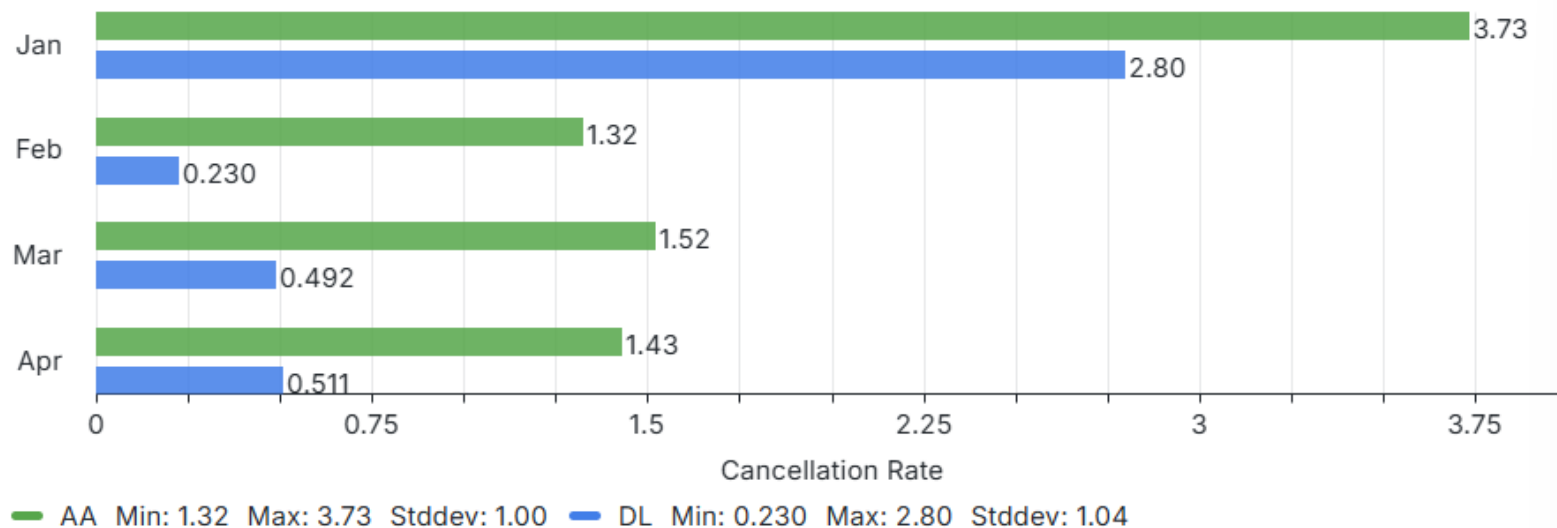
04 Risultati

Monthly Average Departure Delay: American Airlines vs Delta Airlines



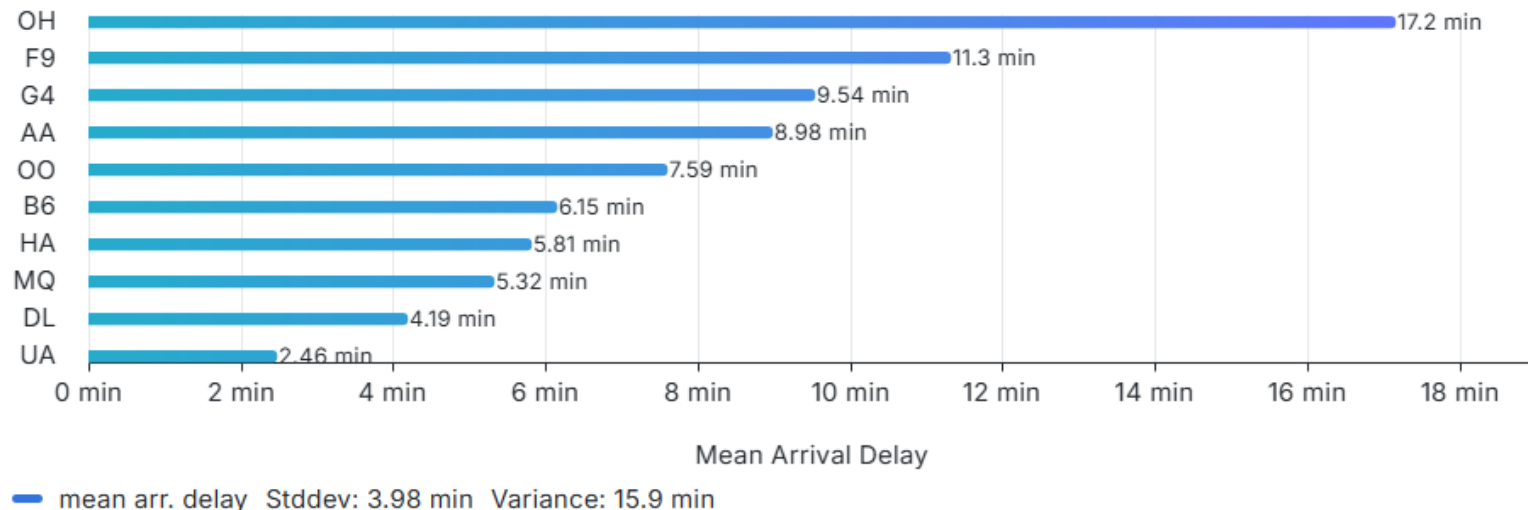
American Airlines supera Delta Airlines in ogni mese (Jan – Feb 2025).

Monthly Average Cancellation Rate: American Airlines vs Delta Airlines

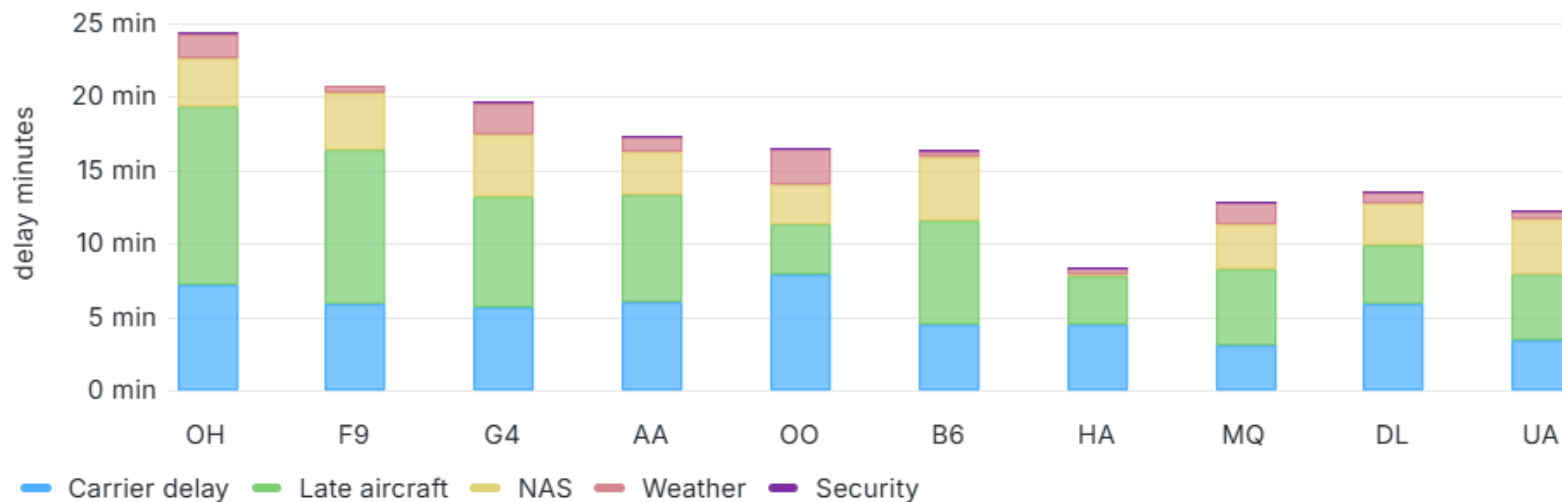


Il **tasso di cancellazione mensile** è considerevolmente **più alto per American Airlines**, più del doppio nel 75% dei casi esplorati.

Top 10 Airlines by Arrival Delay Mean

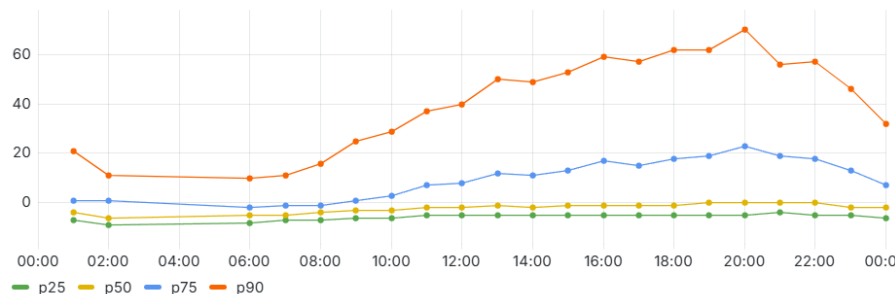


Delay Cause Contributions by Airline Company



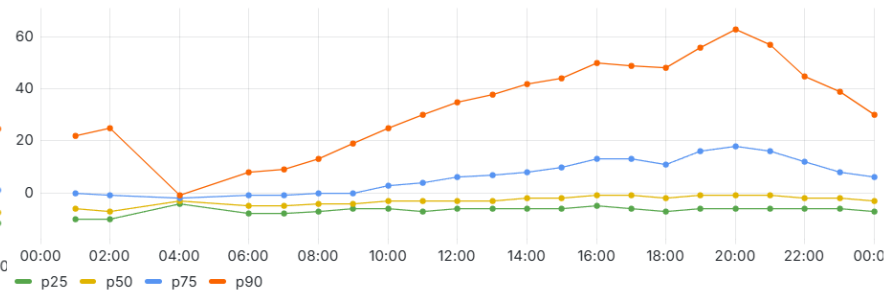
American Airlines: Dep. Delay Percentiles per hour

AA



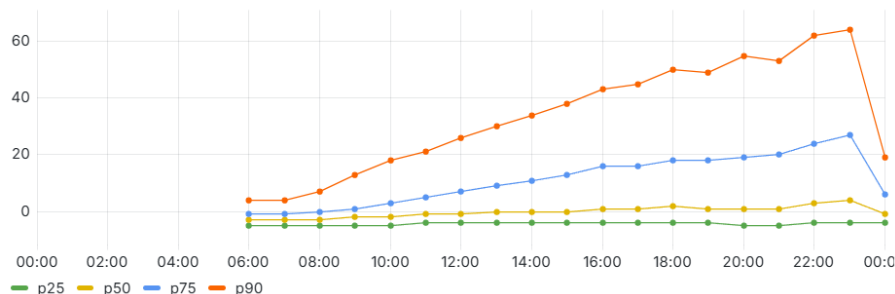
Delta Airlines: Dep. Delay Percentiles per hour

DL



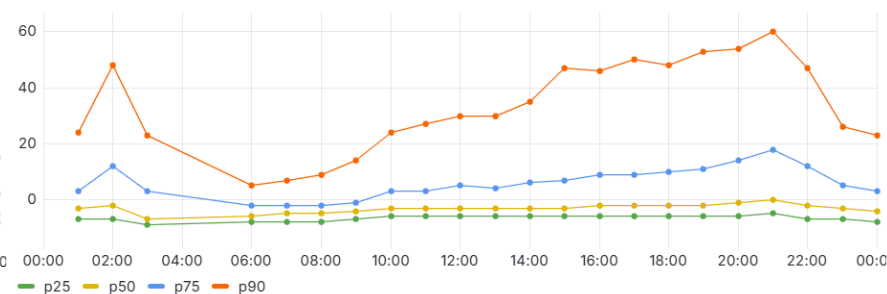
Southwest Airlines: Dep. Delay Percentiles per hour

WN



United Airlines: Dep. Delay Percentiles per hour

UA



05 Analisi delle Prestazioni

Loading time

Lettura Parquet da HDFS (include select per column pruning)

Processing time

Pura computazione Spark (computation_s) — separata da I/O

End-to-end time

Totale: setup + I/O + compute + output + overhead di sistema

E1 DataFrame vs RDD

Tutte e 3 le query · 1 worker · 20 iterazioni (5 warm-up + 15 valide)
Osserviamo end-to-end e processing time

E2 Cold Start vs Warm Start

Cold: Spark session ricreata ad ogni iterazione (0 warm-up)
Warm: Session condivisa (5 warm-up escluse)

E3 Scalabilità Worker

1 → 4 worker Spark (1 core, 1 GB RAM ciascuno) · 20 iterazioni per configurazione

E4 Impatto del Caching

.cache() su ogni fase vs cache solo dove necessario (riuso effettivo).
Effetto su loading/processing/output

Hardware: AMD Ryzen 7 5700U · 8 core · 16 GB RAM | Deploy locale: WSL2 + Docker Compose

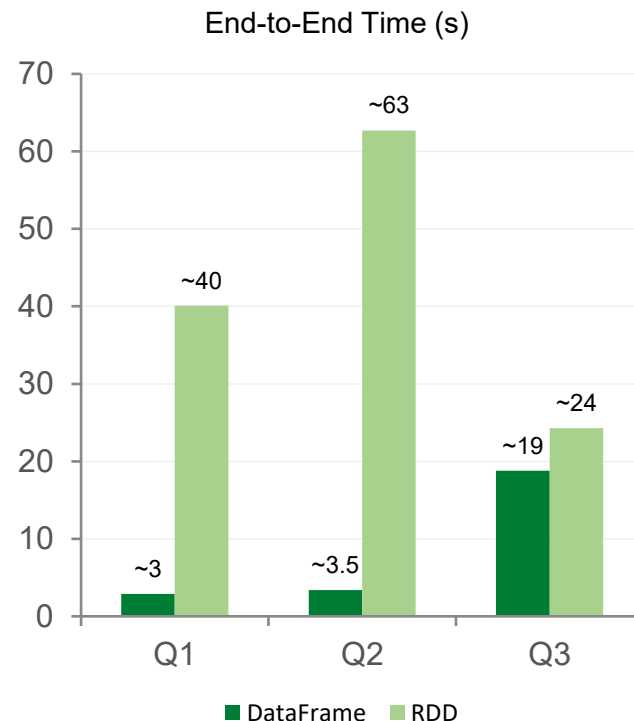
End-to-End Time (secondi) – 1 Worker (5 iterazioni warm-up escluse + 15 valide)

Query	DataFrame	RDD	Speedup DF
Q1	2.91 s	40.09 s	×13.8
Q2	3.38 s	62.70 s	×18.5
Q3	18.78 s	24.27 s	×1.3

Perché il divario è così ampio?

- **Catalyst Optimizer:** column pruning, predicate pushdown, shuffle ottimizzato
- **RDD API:** serializzazione Java, nessuna ottimizzazione automatica
- **Q3 gap ridotto:** t-digest evita il collect dei dati raw ma introduce overhead driver-executor

=> DataFrame superiore in tutti gli scenari grazie al Catalyst Optimizer



05 – E2: Cold vs Warm Start (DataFrame)

Cold start (Spark session ricreata ad ogni iterazione): 15 iterazioni validi

Warm start (Session condivisa): 5 iterazioni warm-up (escluse) + 15 iterazioni valide

Query 1:

Fase	Cold (s)	Warm (s)
Loading	14.60 ± 1.75	0.55 ± 0.04
Processing	6.16 ± 0.95	0.77 ± 0.07
End-to-end	25.37 ± 4.43	2.86 ± 0.45

Query 2:

Fase	Cold (s)	Warm (s)
Loading	13.94 ± 0.36	0.45 ± 0.03
Processing	7.73 ± 0.31	0.79 ± 0.07
End-to-end	26.36 ± 0.59	2.83 ± 0.20

Query 3:

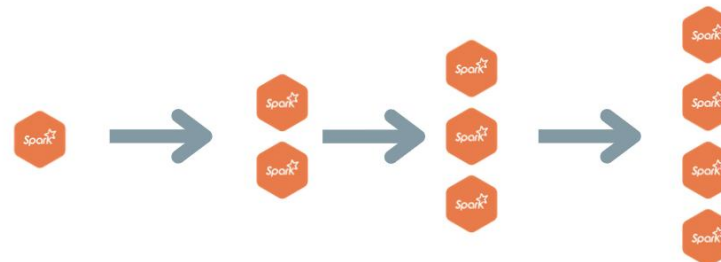
Fase	Cold (s)	Warm (s)
Loading	14.20 ± 0.43	0.39 ± 0.04
Processing	15.20 ± 0.55	8.05 ± 0.19
End-to-end	41.41 ± 1.19	15.14 ± 0.36

Impatti principali:

- **Loading:** OS/JVM file cache assente in cold start
- **Processing:** JIT compilation non ancora calda
- **StdDev** cold: 4.43s vs warm: 0.45s
- **Overhead:** Spark setup
- **Instabilità** del primo accesso a disco

Configurazione:

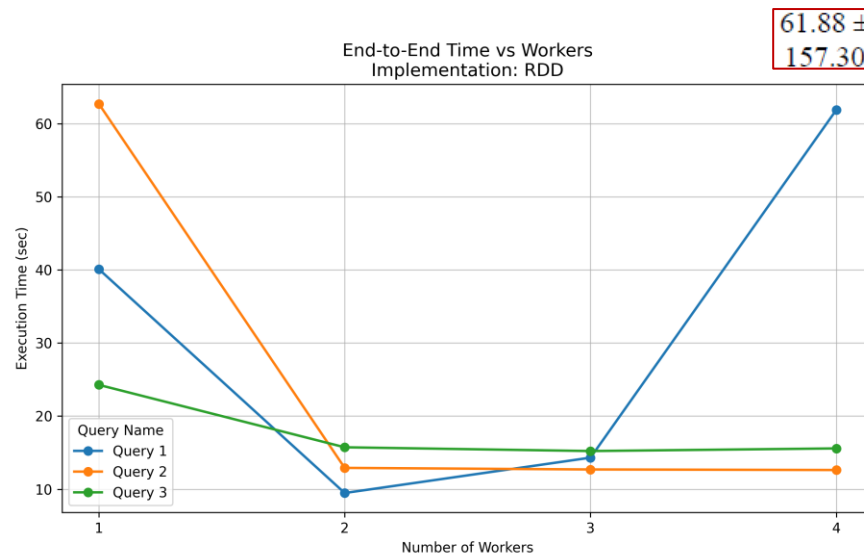
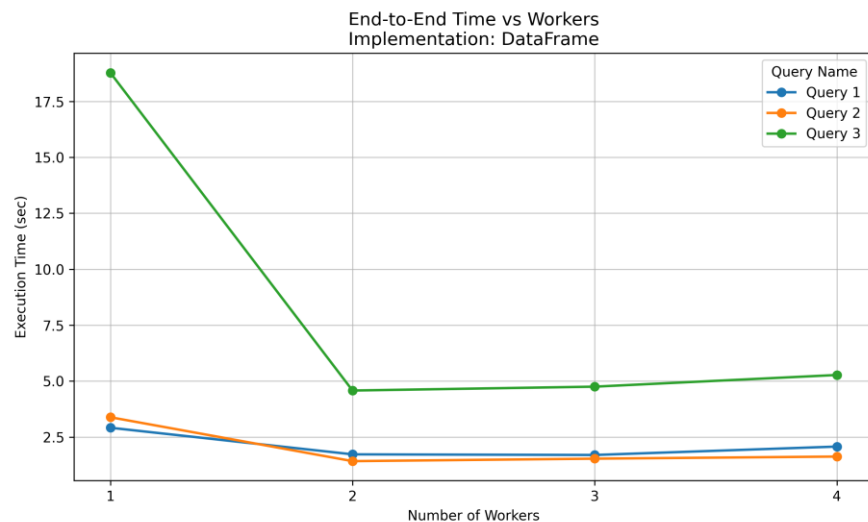
- Numero di Worker: 1 – 4
- Risorse per Worker:
 - 1 core;
 - 1 GB RAM;
- Esecuzione locale (WSL2, Docker Compose);
- 20 iterazioni totali;
- 5 iterazioni di warm-up (scartate);
- API Dataframe vs API RDD;



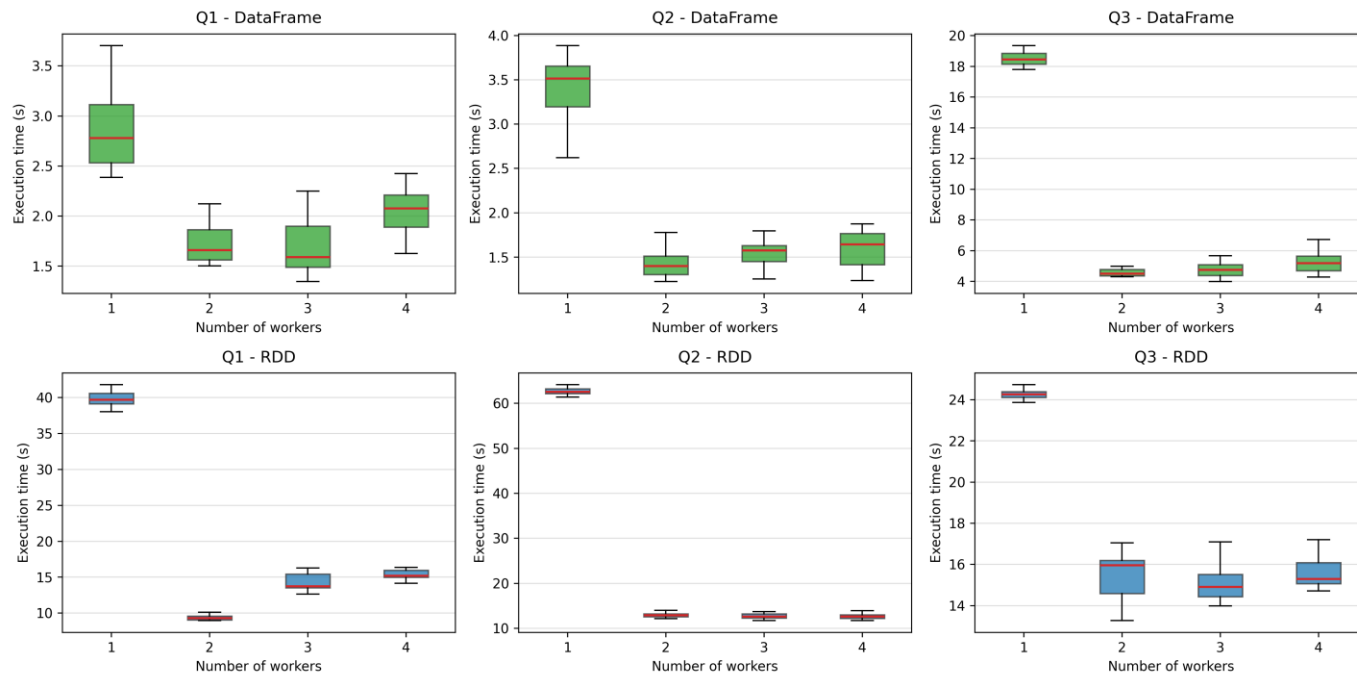
Confronto tempo end-to-end: API Dataframe vs API RDD

Oltre i **2 worker** il guadagno tende a stabilizzarsi o peggiorare per effetto dell'overhead dell'ambiente locale containerizzato.

La presenza di **outliers** (deployment locale) ha avuto un forte **impatto sulla media** dei tempi che per questo **non è rappresentativa** degli effetti dello scaling.



End-to-end execution time distribution



- N = 2 ottimale;
- Parallelismo migliora performance;
- RDD guadagno maggiore;
- Più worker → **contesa delle risorse** (RAM, CPU, datanode), **overhead di comunicazione** (shuffling);

Query 1 (API Dataframe): confronto tra implementazione scelta e implementazione con `.cache()` ad ogni fase

`cache` = `.cache()` su tutti i dataframe intermedi per la separazione dei tempi

Fase	Impl.	Mean (s)	StdDev
Loading	cache	2.36	0.23
Loading	no cache	0.55	0.04
Processing	cache	0.43	0.06
Processing	no cache	0.77	0.07
Output	cache	0.65	0.25
Ouput	no cache	0.89	0.36
End-to-end	cache	4.37	0.53
End-to-end	no cache	2.86	0.45

Vantaggi

- **Separare** in modo **accurato** loading, computation e output;
- Riduzione dei tempi delle fasi successive;

Svantaggi

- **Aumento dei tempi** (materializzazioni aggiuntive);
- Aumenta la **pressione sulla memoria**;
- Ostacolo all'ottimizzazione del piano da parte di Spark (**Catalyst Optimizer**).

Soluzione: `cache()` è stata quindi mantenuta solo dove giustificata da un riutilizzo effettivo del `DataFrame`.

06 Conclusioni

1. L'**API DataFrame** riporta in maniera consistente **performance migliori** rispetto all'API RDD;
→ Catalyst Optimizer, *column pruning*, *predicate pushdown* e pianificazione efficiente degli shuffle;
2. La differenza tra performance iniziali e **performance a regime** per una corretta valutazione della pipeline;
→ caching della Spark Session, primi accessi ai dati e al riscaldamento della JVM;
3. Il **vantaggio dello scaling** dipende fortemente da:
 1. Ambiente di deploy;
 2. Implementazione: *wide transformations* e *shuffling*;
4. Persistenza esplicita:
 1. Limiti dell'accuratezza della misurazione dei tempi;
 2. **Compromesso** tra **misurazione corretta** dei tempi delle varie fasi e misurazione dell'**implementazione realistica**;

Limiti e possibili sviluppi futuri

- Deployment distribuito su più nodi con servizi Cloud → *shared nothing*;
- Incremento del numero di datanode HDFS per una migliore *fault tolerance*;

Grazie per l'attenzione!

SABD A.A. 2025/26 | Prof. Cardellini & Nardelli

Valentina Jin – valentina.jin@students.uniroma2.eu

Elisa Marzioli – elisa.marzioli@students.uniroma2.eu